

ARCEC-3D

Manual Técnico

Cómo agregar nuevas funciones matemáticas

Guía para desarrolladores y estudiantes

Departamento de Ciencias Computacionales · CENIDET

1. Introducción

ARCEC-3D interpreta las expresiones algebraicas de los archivos de resultados usando un motor matemático propio. Ese motor reconoce un conjunto definido de funciones (por ejemplo `add`, `sin`, `tanh`) que están declaradas en un único lugar del código, llamado **catálogo de funciones**.

Si un estudiante o investigador necesita que el sistema reconozca una función que hoy no está soportada, basta con agregarla a ese catálogo. Este manual explica cómo hacerlo paso a paso, qué reglas seguir y cómo verificar que quedó bien.

¿Por qué es tan sencillo?

Todo el sistema (la detección de columnas del archivo, el evaluador de expresiones, las gráficas 2D y las superficies 3D) consulta el mismo catálogo. Por eso, al agregar una función ahí, queda disponible automáticamente en toda la aplicación: **no hay que modificar el frontend, la base de datos ni ningún otro archivo**.

2. Dónde está el catálogo de funciones

El archivo que hay que editar es:

```
backend/src/Utils/math.evaluator.js
```

Dentro de ese archivo, busca el objeto llamado `CATALOGO_FUNCIONES`. Se ve así (fragmento):

```
export const CATALOGO_FUNCIONES = {  
  add: (a, b) => a + b,  
  sub: (a, b) => a - b,  
  mul: (a, b) => a * b,  
  sin: (a) => Math.sin(a),  
  cos: (a) => Math.cos(a),  
  tanh: (a) => Math.tanh(a),  
  ...  
}
```

3. Cómo agregar una función nueva

- 1 **Abre el archivo del catálogo.** Abre `backend/src/Utils/math.evaluator.js` en tu editor de código y localiza el objeto `CATALOGO_FUNCIONES`.
- 2 **Agrega una línea con tu función.** Escribe el nombre de la función **en minúsculas**, seguido de dos puntos y la operación matemática correspondiente. Recuerda terminar con una coma.
- 3 **Guarda el archivo.** Guarda con `Ctrl+S`. Si el servidor está corriendo con `npm run dev`, se reinicia solo; si usas `npm start`, deténlo con `Ctrl+C` y vuelve a arrancarlo.
- 4 **Verifica que funcione.** Sube un archivo que contenga una expresión con tu función nueva y comprueba que el sistema la detecte y la grafique sin mostrar el error "Función no soportada".

Ejemplo completo

Supongamos que se necesita agregar el seno hiperbólico (`sinh`) y la potencia (`pow`). El catálogo quedaría así:

```
export const CATALOGO_FUNCIONES = {
  add: (a, b) => a + b,
  sub: (a, b) => a - b,
  ...

  // --- Funciones agregadas ---
  sinh: (a) => Math.sinh(a),
  pow: (a, b) => Math.pow(a, b),
}
```

A partir de ese momento, el sistema podrá interpretar expresiones como `sinh(x1)` o `pow(x1, x2)`.

4. Reglas que debes respetar

Regla	Explicación
Nombre en minúsculas	El sistema convierte todo a minúsculas antes de buscar la función. Escribe la clave en minúsculas; el usuario podrá escribirla como quiera (Sinh, SINH, sinh) y funcionará igual.
Número de argumentos	Debe coincidir con el uso real: una función de un argumento se declara (a) => ..., y una de dos argumentos (a, b) => ...
No usar nombres repetidos	Si el nombre ya existe en el catálogo, la nueva declaración sobrescribe a la anterior y puede alterar resultados previos.
Devolver siempre un número	La función debe regresar un valor numérico. Si puede producir NaN o Infinito, conviene protegerla (ver capítulo 5).
Terminar con coma	Cada línea del catálogo termina con coma, ya que es un objeto de JavaScript.

5. Funciones que pueden fallar matemáticamente

Los algoritmos evolutivos generan expresiones de forma aleatoria, así que es normal que aparezcan divisiones entre cero, raíces de números negativos o logaritmos de cero. Para evitar que la gráfica se rompa, esas funciones se “protegen” devolviendo un valor de penalización en lugar de un resultado inválido.

El catálogo ya incluye varias funciones protegidas (p_log, p_sqrt, p_exp, divide). Si tu función nueva puede fallar, síguelas como ejemplo:

```
// Ejemplo: raíz cuadrada protegida (nunca recibe negativos)
p_sqrt: (a) => Math.sqrt(Math.abs(a)),
```

```
// Ejemplo: división protegida (evita dividir entre cero)
divide: (a, b) => (Math.abs(b) < 1e-9 ? PENALTY : a / b),
```

```
// Ejemplo: logaritmo protegido
p_log: (a) => {
  const v = Math.abs(a)
  return v < 1e-9 ? PENALTY : Math.log(v)
},
```

PENALTY es una constante que ya existe en el archivo (vale 1e9). El sistema la interpreta como “valor inválido” y la descarta al dibujar la gráfica, evitando picos que deformen la superficie.

6. Cómo verificar que quedó bien

Puedes comprobarlo de dos maneras, de la más rápida a la más completa:

Opción A — Desde la aplicación

Prepara un archivo CSV de prueba con una sola columna que contenga tu función, por ejemplo:

```
T1  
sinh(x1)
```

Súbelo en la pantalla “Graficar”. Si el sistema detecta la columna y muestra la variable x_1 en el panel, la función quedó registrada correctamente.

Opción B — Probando el endpoint directamente

Con una herramienta como Postman, envía una petición al servidor:

```
POST http://localhost:3001/api/archivos/variables  
Body (JSON):  
{  
  "expresiones": ["sinh(x1)"]  
}
```

Si responde con la lista de variables detectadas y sin errores, la función funciona.

7. Problemas comunes

Síntoma	Causa probable y solución
Aparece "Función no soportada"	El nombre no coincide con la clave del catálogo, o el archivo no se guardó. Revisa la ortografía y que el servidor se haya reiniciado.
El servidor no arranca tras editar	Probablemente falta una coma o un paréntesis en la línea agregada. Revisa el mensaje de error en la terminal, que indica la línea exacta.
La gráfica sale plana o con picos enormes	La función devuelve valores inválidos (NaN o Infinito). Protégela como se explica en el capítulo 5.
La columna del archivo no se detecta	El sistema solo reconoce como función las columnas cuyo contenido incluya al menos una llamada de función. Un valor suelto (un número o un identificador) se toma como metadato.